

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a1fb6cf73ace377c5.jsonl`  
- SHA-256: `c371a9813802b9fdf8c56d1123e2d139d6690222fe652cc75f616897cd6f535e`  
- Source modified: `2026-07-02T03:02:46+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T03:01:14.652Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.

A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer:

```
{  
  "title": "scratch/ is gitignored as 'never meant to be committed' yet 7 scripts are still tracked",  
  "category": "hygiene",  
  "severity": "P2",  
  "file": ".gitignore",  
  "line": 71,  
  "evidence": ".gitignore:69-71 'Throwaway repro scripts + PR-body / handoff scratch notes ... never meant to be committed' + `scratch/`; but `git ls-files scratch` returns 7 files (scratch/analyze_motion.py, clean_state.py, diagnose_rally.py, gemini_video_debugger.py, list_models.py, test_ai_segmenter.py, test_thresholds.py), tracked since ~2026-05-18 (commit 72f972e). Checked: none contain literal keys. artifacts/ by contrast is clean (ignored AND untracked), so #377's fix is otherwise holding.",  
  "why_it_matters": "Tracked-but-ignored files are a trap: edits to them silently appear in git status and can ride into commits, and the dir's stated contract (PR bodies/handedoff notes never committed) is what keeps future secret-ish scratch content out of history.",  
  "proposed_fix": "git rm --cached the 7 scratch/*.py files (promote any keeper to tools/ first); one-line follow-up in the cleanup audit doc.",  
  "effort": "S",  
  "tracked_in": ""  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training, product/strategy/licensing, real cloud spend, counsel ToS/DPA?

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code, say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T03:01:23.579Z)

Already up to date.

master

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)

3. user (2026-07-02T03:01:29.931Z)

Coverage data

.coverage

.coverage.*

htmlcov/

.pytest_cache/

Local Claude Code config (machine-specific)

.claude/

Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from ruff + pytest; never meant to be committed -- keep them out of `git status` too).

scratch/

.venv/

Private leak-scan name-list (real player/venue names for LOCAL `tools.leak\_scan --name-scanning` -- NEVER committed; CI runs the name-free structural guard).

.leakscan_private

Stray scheme-prefixed report dirs from a remote URI accidentally resolved as a literal path (gs://b/x -> ./gs://b/x). Root cause fixed in #397; this is a belt-and-braces guard so regression can never be committed. Anchored to the repo root.

```
---
scratch/analyze_motion.py
scratch/clean_state.py
scratch/diagnose_rally.py
scratch/gemini_video_debugger.py
scratch/list_models.py
scratch/test_ai_segmenter.py
scratch/test_thresholds.py
---
exit=1
```

4. assistant (2026-07-02T03:01:39.076Z)

The check-ignore exit=1 is unexpected ? let me dig into whether the ignore rule actually applies.

5. user (2026-07-02T03:01:41.956Z)

Exit code 1

.gitignore:71:scratch/ scratch/definitely_new_file.py

exit=0

69:# Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from

71:scratch/

ABLATION_FRAMEWORK_PLAN.md

GATE_AB_NUMBERS_AND_IMPLICATIONS.md

HIGH_RISK_REMEDIATION_PLAN.md

OWNED_MODEL_PHASE0_PLAN.md

PACKAGING_PLAN.serving-wip.md

PERSONALIZATION_FEATURE_DESIGN.md

PERSONALIZATION_PIVOT_ANALYSIS.md
PR1_BODY.md
PR1_HANDOFF.md
PR2_HANDOFF.md

6. user (2026-07-02T03:01:58.254Z)

8615534 refactor: adopt approved layered directory structure
5257fd5 feat: pipelined YOLO chunk processing & unified Gemini model config
6b7d5dd Refactor Gemini prompt to file top, add defensive timestamp parsing, add compiler logging, and create scratch debug/cleanup tools
0f76429 Refactor rally detection to pluggable base and registry, integrate Gemini 2.5 Flash segmenter, add local fast ffmpeg trimming, and update developer docs
72f972e Checkpoint: Add debug trim feature, runner script, and motion analysis scratch script

6b7d5dd Refactor Gemini prompt to file top, add defensive timestamp parsing, add compiler logging, and create scratch debug/cleanup tools
0f76429 Refactor rally detection to pluggable base and registry, integrate Gemini 2.5 Flash segmenter, add local fast ffmpeg trimming, and update developer docs
72f972e Checkpoint: Add debug trim feature, runner script, and motion analysis scratch script

aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
959c630 fix(#397): write per-video report to the resolved local path, not the raw URI (#400)
71a8241 chore(repo): fix corrupted .gitignore (UTF-16 tail) + ignore artifacts/, track product overview (#377)
5c2eec8 chore(security): P0 (safe scope) ? scrub personal paths from code + add leak-scan CI guard (#314)
c8e60ee fix(gen0): provenance-stamp the manifest git_sha off a git-archive tarball (#269)

72f972e 2026-05-18 Checkpoint: Add debug trim feature, runner script, and motion analysis scratch script

scratch/analyze_motion.py | 72 +++
1 file changed, 72 insertions(+)

7. assistant (2026-07-02T03:02:14.159Z)

All core claims verified. Quick spot-check on the secret-scan claim and current status of the tracked files.

8. user (2026-07-02T03:02:16.669Z)

```
e410136:scratch/gemini_video_debugger.py:32:    api_key = os.environ.get("GEMINI_API_KEY")
e410136:scratch/gemini_video_debugger.py:33:    if not api_key:
e410136:scratch/gemini_video_debugger.py:34:        print("[ERROR] GEMINI_API_KEY environment
variable is not set!")
e410136:scratch/gemini_video_debugger.py:35:        print("Please set it in your PowerShell:
$env:GEMINI_API_KEY=\"your_api_key\"")
e410136:scratch/gemini_video_debugger.py:39:    client = genai.Client(api_key=api_key)
e410136:scratch/list_models.py:6:api_key = os.environ.get("GEMINI_API_KEY")
e410136:scratch/list_models.py:7:if not api_key:
e410136:scratch/list_models.py:8:    print("[ERROR] GEMINI_API_KEY is not set.")
e410136:scratch/list_models.py:11:client = genai.Client(api_key=api_key)
e410136:scratch/test_ai_segmenter.py:54:    api_key = os.environ.get("GEMINI_API_KEY")
grep-exit=0
7
```

9. user (2026-07-02T03:02:46.015Z)

Structured output provided successfully